

PROJECT 1 — INTELLIGENT MEDICAL FRONT-DESK ASSISTANT

A complete, realistic agentic LLM project

1. Problem Scenario

A multi-speciality clinic chain wants to deploy an AI assistant that can:

- Talk to patients conversationally
- Ask relevant symptom-based questions
- Retrieve medical FAQs, clinic policies, and department details
- Identify the correct department or specialist
- Suggest appointment slots
- Summarize conversations for doctors
- Handle uploaded medical reports (PDFs)
- Provide safe general-health guidance

The assistant must behave like a **real front-desk staff member**, working with clinic data, policies, schedules, and tools.

The system must be built using:

- **Ollama + Mistral**
 - **LangChain**
 - **RAG**
 - **Memory**
 - **Streamlit frontend**
 - **FastAPI backend**
 - **Tool integrations**
 - *(Bonus)* MCP, Agents, Caching, Autonomous logic
-

2. Functional Requirements

A. Core Features

1. Multi-turn conversation with patients
 2. Collect:
 - Name
 - Age
 - Symptoms
 - Duration of symptoms
 - Severity
 3. Retrieve accurate medical FAQ answers through RAG
 4. Identify department (ENT, Cardiology, Dermatology, etc.)
 5. Retrieve and display:
 - Department working hours
 - Contact
 - Doctor availability
 6. Trigger appointment suggestion tool
 7. Summarize the encounter for doctors
 8. Support PDF upload (lab report, prescription)
-

B. RAG Requirements

Students must implement:

- Chunking
- Embedding using an Ollama embedding model or HuggingFace sentence-transformers
- Vector store:
 - **ChromaDB or FAISS** (they must justify)
- Retrieval pipeline

Documents to store in vector DB:

- Clinic policies
- Department descriptions
- Common symptoms → department mapping
- Health FAQs
- Pre-visit instructions

C. Memory Requirements

Students must use:

✓ Entity Memory

To store:

- Patient name
- Age
- Symptoms
- Medical history provided during chat

✓ **Summary Memory**

To compress long interactions.

✓ **Optional:**

- Long-term vector memory for repeat users (bonus)

Students must justify their memory choices.

D. Tools / Functions Students Can Build

Students must integrate at least **three** tools.

Examples:

1. Appointment Scheduling Tool

Input: department, date

Output: available slots

Implementation: JSON file or SQLite

2. Department Info Tool

Returns:

- Location
- Timings
- Phone
- Consultation fee

3. PDF Reader Tool

Extract text from uploaded medical reports.

4. Symptoms → Department Classifier

A simple ruleset or small ML model.

5. Drug Interaction Checker (optional)

A mocked database:

- paracetamol + xyz → okay
- ibuprofen + abc → caution

6. Emergency Triage Tool (optional)

If patient mentions red-flag symptoms:

- severe chest pain
- loss of consciousness
- bleeding

Provide immediate guidance.

E. APIs They May Integrate

Students can choose any of the following:

General Health Information APIs

- <https://health.gov/myhealthfinder/api/>
- <https://clinicaltables.nlm.nih.gov/>

PDF OCR APIs

- <https://www.ocr.space/OCRAPI>

Appointment Scheduling Mock API

Students build their own small internal API.

Drug Info APIs

- <https://rxnav.nlm.nih.gov/REST/>

Medical Dictionary APIs

- <https://dictionaryapi.dev/>
-

F. Agentic AI & Autonomous Agent Logic (Bonus)

Agent Roles

1. **Patient Intake Agent**
 - Collect patient info
 - Ask clarifying questions
2. **Medical Retrieval Agent**
 - Calls RAG
 - Fetches relevant medical FAQs
3. **Department Routing Agent**
 - Uses rules + RAG to map symptoms to department
4. **Appointment Agent**
 - Calls the scheduling tool
5. **PDF Interpretation Agent** (optional)
 - Reads reports and extracts abnormalities

Autonomous Logic Ideas

- The agent keeps asking for symptoms until confidence > 0.6
 - If red-flag symptoms appear → trigger emergency protocol
 - If patient uploads a report → automatically interpret + summarize
-

G. Caching Strategy (Bonus)

Students may use:

1. LangChain SQLite Cache

Cache:

- Repeated FAQs
- Department timetables

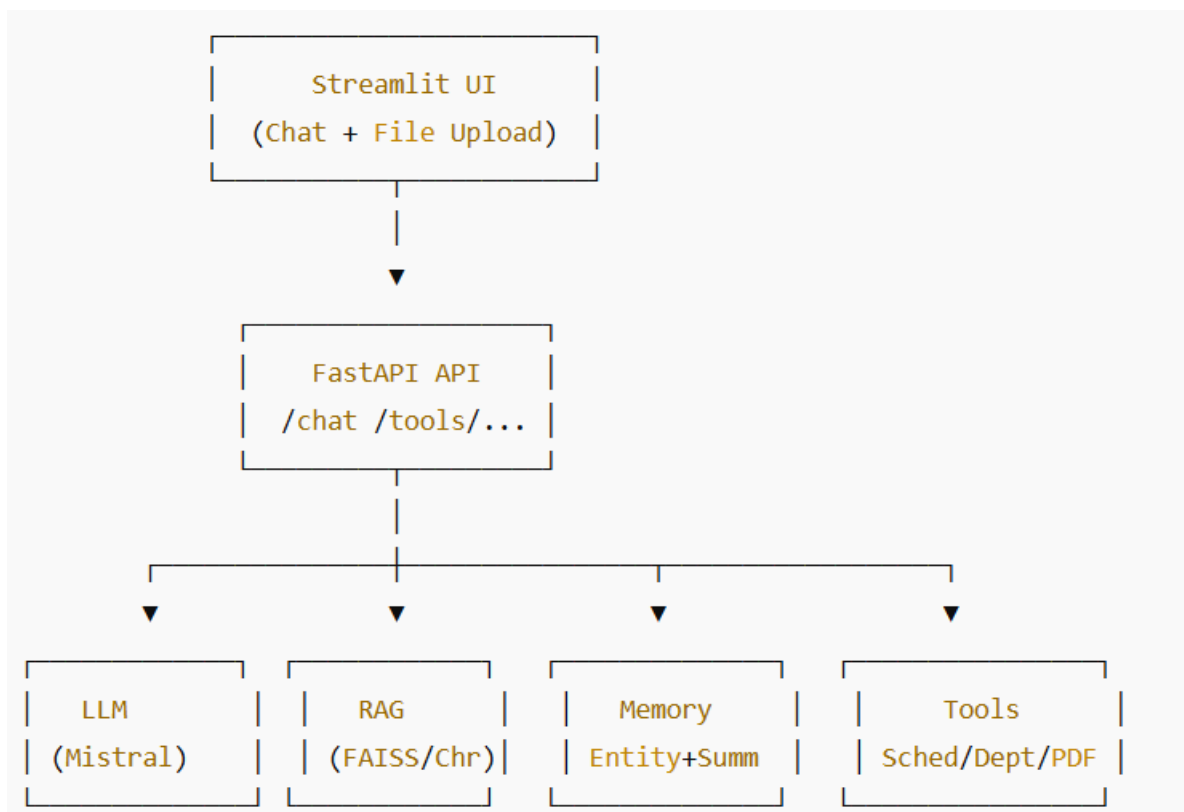
2. In-memory caching

For repeated RAG queries.

They must explain:

- Why caching reduces latency
- What data is safe to cache
- Expiration strategy

3. Architecture Diagram



4. Folder Structure

```
medical-assistant/  
|  
├─ backend/  
|   ├── main.py (FastAPI)  
|   ├── tools/  
|   |   ├── schedule.py  
|   |   ├── departments.py  
|   |   └─ pdf_reader.py  
|   ├── rag/  
|   |   ├── vector_store.py  
|   |   └─ retriever.py  
|   ├── memory/  
|   └─ models/  
|  
├─ frontend/  
|   └─ app.py (Streamlit)  
|  
├─ evaluation/  
|   ├── rag_tests.ipynb  
|   ├── bleu_rouge.ipynb  
|   └─ latency_tests.ipynb  
|  
└─ data/  
    ├── clinic_policies/  
    ├── department_info/  
    └─ health_faqs/
```

5. Evaluation Tasks

A. RAG Evaluation Notebook

Using 5–10 medical queries:

- Precision@k
- Recall@k
- Top-k hit rate
- Failure cases + analysis

B. Response Quality Tests

- ROUGE-L
- BLEU
- BERTScore

Using a reference set of curated FAQ answers.

C. Latency Tests

Measure:

- LLM response time
- Retrieval time
- End-to-end latency

D. Stress Tests

- 50 parallel requests
- 20-turn long conversation stability

E. Error Handling Tests

- Missing symptoms
- Invalid input
- Bad PDF

6. Deployment Requirements

A. FastAPI Backend

- `/chat`
- `/schedule`
- `/department`
- `/upload`
- `/rag/query`
- Supports async
- Returns JSON

B. Streamlit Frontend

- Chat window
- PDF upload section
- Department suggestion display
- Retrieved document viewer
- Appointment scheduling UI

7. Student Submission Checklist

Students must provide:

- Architecture explanation
- Justification for:
 - vector store
 - RAG type
 - memory types
 - tool design
 - caching strategy
- Streamlit UI / FastAPI backend
- Evaluation notebooks
- Test results
- Error handling demonstration
- Optional agentic/MCP integration